

Docker Container Security

By –Prakash

Khasor,
Assistant Professor(Cyber Security),
Institute of Forensic Science,
Gujarat Forensic Sciences University

Introduction

- Understanding Docker and its need
- Compare with VM
- Security features and Security Auditing

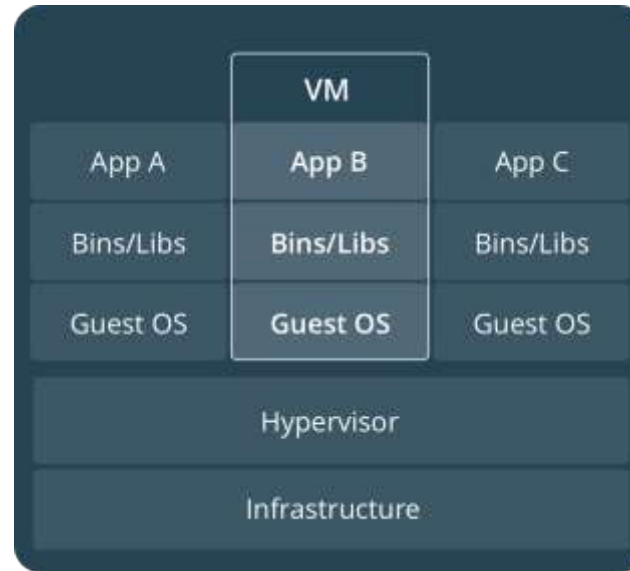
What is Docker?

- Docker is a containerization platform that packages your application and all its dependencies together in the form of a docker container to ensure that your application works seamlessly in any environment.
- Docker Container is a standardized unit which can be created on the fly to deploy a particular application or environment. It could be an Ubuntu container, CentOS container, etc. to full-fill the requirement from an operating system point of view. Also, it could be an application oriented container like CakePHP container or a Tomcat-Ubuntu container etc.

Comparing Containers and VMs



Containers are an
app level construct



VMs are an
infrastructure level
construct to turn
one machine into
many servers

Key Benefits of Docker Containers

- **Speed:-**

- No OS Boot time

- **Portability:-**

- Less Dependencies between Process Layers

- **Efficiency:-**

- Less OS overhead

Docker Basics

● **Image**



● **Container**



● **Engine**



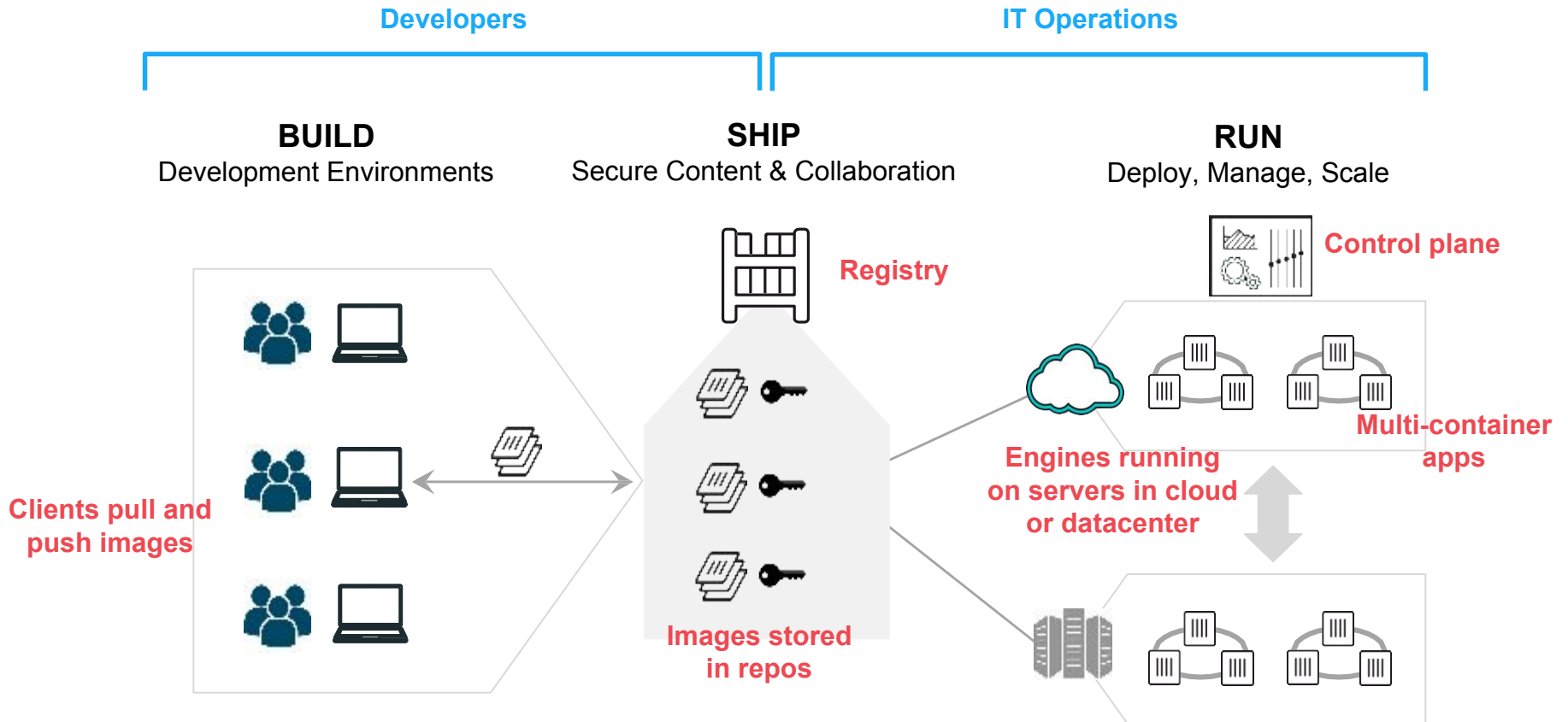
● **Registry**



● **Control Plane**



Containers as a Service



Building a Secure Supply Chain

Container App Lifecycle Workflow

Private Image Registry	Secure Access and User Management	Application and Cluster Management
Image Scanning and Monitoring	Content Trust and Verification	Policy Management
Security	Network	Volumes
Distributed State	Container Runtime	Orchestration



Enterprise Edition



Docker Engine



Usable
Security



Trusted
Delivery



Portable

Container Security features

- **Namespaces**:- PID, Mount, Network, IPC, UTC, User.
- **Cgroups** :- Limit CPU, Memory, IO
- **Capabilities** :- Reduced root access. 36 capabilities to control as of kernel 3.19.0.21.
- **Seccomp profiles** :- Control kernel system calls. 300+ system calls available that can be controlled using these profiles.
- Special kernel modules like **AppArmor**, **SELinux** – Provides granular control over Kernel resources.

Namespaces – PID, Mount

- `docker run -ti --name ubuntu1 -v /usr:/ubuntu1 ubuntu bash`
- `docker run -ti --name ubuntu2 -v /usr:/ubuntu2 ubuntu bash`

PID namespace:

Ubuntu1 Container:

```
root@3a1bf12161c9:/# ps
```

```
PID TTY TIME CMD
```

```
1 ? 00:00:00 bash
```

```
15 ? 00:00:00 ps
```

Ubuntu2 Container:

```
root@8beb85abe6a5:/# ps
```

```
PID TTY TIME CMD
```

```
1 ? 00:00:00 bash
```

```
14 ? 00:00:00 ps
```

Host:

```
$ ps -eaf|grep root | grep bash
```

```
root 5413 1697 0 05:54 pts/28
```

```
00:00:00 bash
```

```
root 5516 1697 0 05:54 pts/31
```

```
00:00:00 bash
```

Mount namespace:

Ubuntu1 Container:

```
root@3a1bf12161c9:/# ls /
```

```
bin dev home lib64 mnt proc run
```

```
srv tmp usr
```

```
boot etc lib media opt root sbin
```

```
sys ubuntu1 var
```

Ubuntu2 Container:

```
root@8beb85abe6a5:/# ls /
```

```
bin dev home lib64 mnt proc run
```

```
srv tmp usr
```

```
boot etc lib media opt root sbin
```

```
sys ubuntu2 var
```

Namespaces – Network, UTS

Ubuntu1 Container:

```
root@3a1bf12161c9:/# ifconfig
eth0 Link encap:Ethernet HWaddr
02:42:ac:15:00:02 inet addr:172.21.0.2
Bcast:0.0.0.0 Mask:255.255.0.0 inet6
addr:
fe80::42:acff:fe15:2/64
```

Ubuntu2 Container:

```
root@8beb85abe6a5:/# ifconfig
eth0 Link encap:Ethernet HWaddr
02:42:ac:15:00:03 inet
addr:172.21.0.3
Bcast:0.0.0.0 Mask:255.255.0.0
inet6 addr:
fe80::42:acff:fe15:3/64
```

- With UTS namespace, each Container gets its own hostname and DomainName

Ubuntu1 Container:

```
root@3a1bf12161c9:/#
hostname
3a1bf12161c9
```

Ubuntu2 Container:

```
root@8beb85abe6a5:/#
# hostname
8beb85abe6a5
```

Namespaces - IPC

Ubuntu 1 Container:

Create shared memory:

```
root@3a1bf12161c9:/# ipcmk -M 100
```

Shared memory id: 0

Display shared memory:

```
root@3a1bf12161c9:/# ipcs -m
```

----- Shared Memory Segments -----

key	shmid	owner	perms	bytes	nattch
-----	-------	-------	-------	-------	--------

					status
--	--	--	--	--	--------

0x2fba9021	0	root	644	100	0
------------	---	------	-----	-----	---

Ubuntu 2 Container:

Create shared memory:

```
root@8beb85abe6a5:/# ipcmk -M 100
```

Shared memory id: 0

Display shared memory:

```
root@8beb85abe6a5:/# ipcs -m
```

----- Shared Memory Segments -----

key	shmid	owner	perms	bytes	nattch
-----	-------	-------	-------	-------	--------

					status
--	--	--	--	--	--------

0x1f91e62c	0	root	644	100	0
------------	---	------	-----	-----	---

Namespaces - User

- To use user namespace, Docker daemon needs to be started with “--userns-remap=username/uid:groupname/gid”. Using “default” for username will create “dockremap” user.

Host:

```
$ ps -eaf|grep bash
231072 4080 4040 0 22:45 pts/13
00:00:00 bash
$ cat /proc/4080/uid_map
0 231072 65536
(userid 0 in Container is mapped to
231072 in host)cgroups
```

Ubuntu1 Container:

```
root@3a1bf12161c9:/# id
uid=0(root) gid=0(root) groups=0(root)
```

cgroups

- cgroups is a Linux kernel feature that provides capability to restrict resources like cpu, memory, io, network bandwidth among a set of processes.
- Docker allows to create Containers using cgroup feature which allows for resource control for the specific Container.
- Following is a Container created with user space memory limited to 500m, kernel memory limited to 50m, cpu share to 512, blkio weight to 400. (cpu share and blkio weight are ratios)

```
docker run -it -m 500M --kernel-memory 50M --cpu-shares 512 --blkio-weight 400 --  
name ubuntu1 ubuntu bash
```

Capabilities

- In Linux, root user typically has all privileges enabled. Capabilities allow finer control for the capabilities that can be allowed for root user.
- In Linux 3.19.0.21, there are 36 capabilities. All capabilities can be seen in “/usr/include/linux/capability.h”.
- Docker turns on only 14 capabilities by default for Containers started with default options.
- Capabilities like insert/remove kernel modules, system clock manipulation are blocked.

**Container with raw net capability
turned off:**

```
docker run -ti --name ubuntu1 --cap  
drop=net_raw ubuntu bash
```

Network access blocked:

```
# ping google.com  
ping: icmp open socket: Operation not  
permitted
```

Seccomp

- Linux kernel feature that limits the system calls that a process can make based on the specified profile.
- Docker uses Seccomp to control the system calls that Container can make.
- Docker disables around 44 of 300+ system calls for Containers started with default options

Profile disabling chmod system call:

```
{ "defaultAction":  
  "SCMP_ACT_ALLOW",  
  "syscalls": [ { "name": "chmod",  
    "action": "SCMP_ACT_ERRNO" } ] }
```

Seccomp illustration:

```
$ docker run --rm -it --security-opt  
seccomp:/home/smakam14/seccomp/pr  
ofil  
e.json busybox chmod 400 /etc/hosts  
chmod: /etc/hosts: Operation not  
permitted
```


AppArmor, SELinux

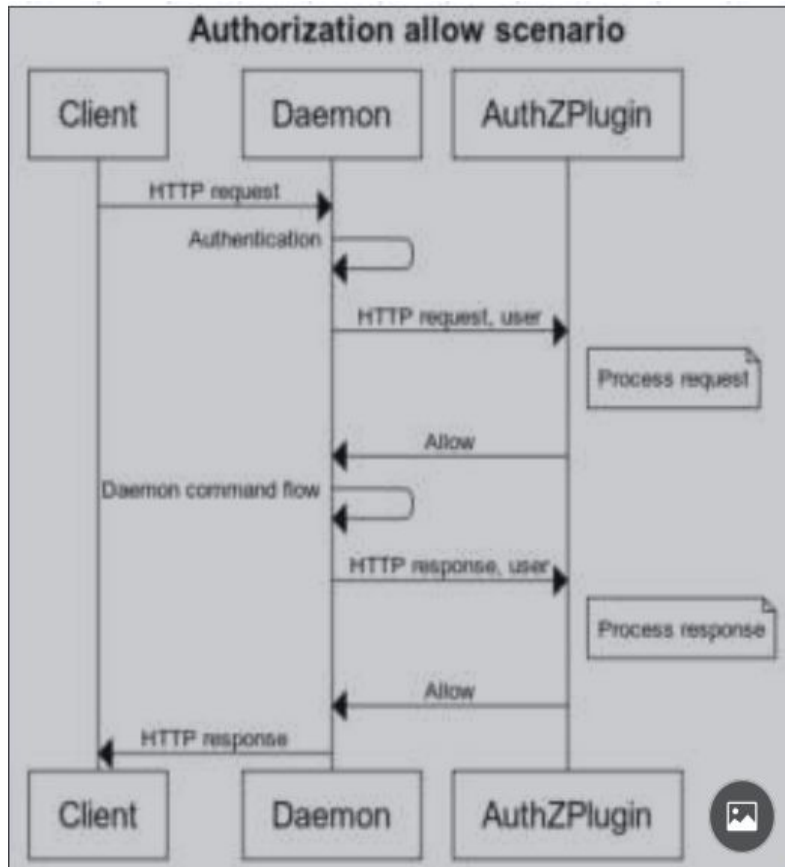
- Both AppArmor and SELinux are kernel modules that gives fine grained control to restrict access to system resources.
- When AppArmor is active for an application, the operating system allows the application to access only those files and folders that are mentioned in its security profile.
- SELinux is a labeling system. Every process, file, directory, network ports, devices has a label assigned to it. We write rules to control the access of a process label to an a object label like a file. The kernel enforces the rules specified in the policy.

Docker Engine Secure access

- Docker engine runs as a daemon and by default listens on the Unix socket, “unix:///var/run/docker.sock”.
- For accessing Docker engine remotely, “http” or “https” using TLS can be used.
- “http” is not advised to be used for security reasons.
- “https” with TLS provides confidentiality, authentication as well as integrity.

Authorization plugin

- Authorization plugin can provide granular access to the Docker daemon based on userid, groupid, command executed, command arguments, time of day etc.
- Twistlock authz broker is one of the first plugins.
- In Twistlock case, policy is created as a JSON file and is given as argument to Docker daemon.
- Docker Data Center and Docker cloud provides RBAC and Multi-tenant support.



Policy.json:

```

{"name":"policy_1","users":[""],"actions":
["contain
er"] , "readonly":true} – Read-only
Container policy
  
```

Starting Docker daemon with auth policy.json:

```

docker run -d --restart=always -v
/var/lib/authz
broker/policy.json:/var/lib/authz-bro
ker/policy.json
-v
/run/docker/plugins/:/run/docker/plu
gins
twistlock/authz-broker
  
```

Container image signing

- Container images need to be signed so that the client knows that image is coming from a trusted source
- Content publisher takes care of signing Container image and pushing it into the registry.
- When content trust is enabled, we can pull only signed images.
- Docker content trust is enabled with “export DOCKER_CONTENT_TRUST=1”.
- root key ,repository key and Yubikey

Container Image scanning

- Docker Security Scan scans Container images and reports vulnerabilities.
- Binary Scan
- Pro-active notification is given to both the publisher and user of Container images.

Biggest Container Security Concern

- Image Security
- Host Security
- Vulnerability
management
- Container Isolation

Container Security Scanners

- **Open Source:-**

- OpenSCAP
- CoreOS Clair
- Anchore

Docker Security Practices

- Have separate containers for each micro-service keeping Container image size small.
- Don't put ssh inside container, "docker exec" can be used to ssh to Container.
- Use only signed Container images.
- Mount devices and volumes as read-only.
- Run application as non-root. If root access is needed, run as root only for limited operations using features like Capabilities, Seccomp, SELinux/AppArmor.
- Keep OS secure with regular updates. Using Container optimized OS is an option here since they provide automatic pushed update.
- Store root keys, passphrase in a safe place and not expose in Dockerfile. Docker has plans to manage keys with Docker datacenter.
- Use Docker official images. These images are curated by Docker so that the highest quality and security is maintained for the official images.
- Use Container security scanning to check for vulnerabilities.
- Use TLS for remote Docker daemon access.

Thank you!